

Nhìn tối ưu hóa thuật toán qua các bài toán đồ thị

Jia You

Trường Trung học số 3 Wugang

2006

Nguồn gốc: A brief analysis of algorithm optimization through graph-theory problems

IOI China candidate team papers / 2006 / Jia You.doc

Tóm tắt

Bài viết lấy các bài toán đồ thị làm đối tượng, lấy tối ưu hóa thuật toán làm chủ đề, và thảo luận theo lối phân loại kết hợp ví dụ. Phần đầu giới thiệu ngắn gọn quan hệ giữa lý thuyết đồ thị và thi tin học. Phần thứ hai phân tích con đường căn bản của tối ưu hóa: tìm ra điểm đặc biệt của bài toán. Phần thứ ba bắt đầu từ việc sửa thuật toán sai, bàn chi tiết một số phương pháp liên quan, qua đó tiếp tục cho thấy việc phát hiện tính đặc thù của bài toán thúc đẩy tối ưu hóa thuật toán như thế nào.

Từ khóa: lý thuyết đồ thị; tối ưu hóa thuật toán; phân tích lỗi.

1 Mở đầu

Lý thuyết đồ thị là một nhánh toán học rất thú vị và có liên hệ mật thiết với thi tin học. Khi số lượng bài toán đồ thị ngày càng tăng, một số mô hình đồ thị kinh điển cùng các thuật toán tương ứng đã trở thành kiến thức không thể thiếu trong thi đấu. Đồng thời, đề bài cũng ngày càng chú trọng tới việc chuyển hóa mô hình và tối ưu hóa thuật toán. Điều đó có nghĩa là trong quá trình biến kiến thức thành điểm số, ta cần bỏ thêm nhiều công sức hơn. Bài viết này chọn tối ưu hóa thuật toán làm chủ đề và thử tổng kết, thảo luận một số vấn đề liên quan.

Ngoài ra, do cơ chế chấm hộp đen, tin học mà ta trải nghiệm có thể nói là một môn học lấy kết quả để luận thành bại. Điều đó tốt, vì kết quả là sự tổng kết của cả quá trình. Tuy vậy, trong một thảo luận lấy tối ưu hóa làm chủ đề, thuật toán tối ưu cuối cùng chỉ dùng để chứng minh rằng quá trình tối ưu của ta là thiết thực và hiệu quả.

2 Tìm điểm đặc biệt: con đường căn bản của tối ưu hóa

2.1 Giới thiệu

Mọi cải tiến làm thuật toán đẹp hơn đều có thể gọi là tối ưu hóa. Nhưng khi xét toàn cục một bài toán, quá trình tối ưu hóa nên bao gồm toàn bộ các cải tiến từ thuật toán ban đầu tới một thuật toán tốt. Đây thường là quá trình từng bước phát hiện và tận dụng các điểm đặc biệt của bài toán, làm thuật toán có tính nhắm đích hơn.

Căn bản của tối ưu hóa tốt là tìm ra điểm đặc biệt của đề. Đây là một ý tưởng rộng, không có bao nhiêu bước cố định hay mẹo vặt. Khi giải một bài cụ thể, ta chỉ có thể dựa vào kinh nghiệm tối ưu hóa

rộng, đủ kiên nhẫn, và một phần cảm hứng. Về kinh nghiệm, một số bài viết trước đã lần lượt giới thiệu quá trình khai thác sâu thông tin ở các lớp bài có đặc trưng chung. Chương này không đi sâu vào một lớp bài cụ thể, mà dùng một thuật toán kinh điển để giải thích ý tưởng “tìm điểm đặc biệt”.

2.2 Ví dụ: ghép cực đại trên đồ thị hai phía

Một bộ ghép của đồ thị là một tập cạnh sao cho không có hai cạnh nào có chung đỉnh. Bài toán ghép cực đại trên đồ thị hai phía yêu cầu tìm một bộ ghép có số cạnh lớn nhất. Cách cơ bản nhất để giải bài này là chuyển nó thành một mô hình luồng mạng.

Để tiện trình bày, gọi hai tập đỉnh của đồ thị hai phía là A và B . Thêm nguồn và đích vào đồ thị; nối nguồn tới mỗi đỉnh của A bằng cạnh có sức chứa 1; nối mỗi đỉnh của B tới đích bằng cạnh cũng có sức chứa 1. Sau đó thêm các cạnh của đồ thị gốc vào mạng dưới dạng cạnh có hướng từ A sang B , sức chứa 1. Trong đồ thị mới, sức chứa giới hạn mỗi đỉnh được phủ bởi nhiều nhất một cạnh và mỗi cạnh chỉ được tính một lần. Dễ thấy giá trị luồng cực đại của mạng bằng số cạnh trong bộ ghép cực đại. Qua thuật toán luồng cực đại, ta cũng có thể thu được phương án chọn cạnh cụ thể.

Một tối ưu hóa hiển nhiên là không ghi sức chứa, vì mọi cạnh đều có sức chứa bằng một. Kể đến, đường tăng trong mạng này rất đặc biệt: nó chắc chắn bắt đầu từ nguồn, đi qua lại một số lần giữa A và B , rồi từ B tới đích. Khi tìm kiếm, ta có thể bỏ qua cạnh đầu nối với nguồn và cạnh cuối nối với đích, trực tiếp bắt đầu từ một đỉnh chưa ghép trong A và kết thúc ở một đỉnh chưa ghép trong B . Có thể hình dung rằng việc giảm hai cạnh khỏi đường mục tiêu của tìm kiếm theo chiều rộng ảnh hưởng rất lớn tới số nút phải mở rộng. Đó chính là tư tưởng cơ bản của thuật toán Hungary.

Độ phức tạp thời gian của thuật toán luồng mạng cơ bản và thuật toán Hungary thực ra không khác nhau, đều là $\mathcal{O}(MN)$; thuật toán nhanh hơn có thể xem trong tài liệu tham khảo. Nhưng thuật toán Hungary có ưu thế tuyệt đối về bộ nhớ, độ khó cài đặt, và thời gian chạy thực tế.

Gần như chắc chắn thuật toán Hungary ban đầu không được tìm ra bằng cách tối ưu hóa như trên. Tuy nhiên, hai thủ pháp ấy rất thực dụng trong thiết kế thuật toán luồng mạng: đơn giản hóa cách lưu trữ theo tính đặc thù của đồ thị, và thiết kế riêng cách tìm đường tăng.

2.3 Ví dụ: quy hoạch nông trại

Người bạn tốt Sealock của Xiao Keke rất thích ăn đậu phộng, nên mượn nông trại của Xiao Keke để làm thí nghiệm chọn giống. Anh ta chia nông trại rất đều đặn thành lưới $M \times N$ ô chữ nhật, với $MN \leq 5000$. Do diện tích mặt nước và đầm lầy ở các ô khác nhau, diện tích thực tế có thể trồng trọt của mỗi ô cũng khác nhau; gọi diện tích có thể trồng của ô (i, j) là $A(i, j)$.

Mỗi ô nhiều nhất chỉ trồng một giống đậu phộng. Ô có diện tích trồng được bằng không không được chọn để thí nghiệm. Đồng thời, để tránh phấn hoa lan sang ô kề cạnh làm sai kết quả thí nghiệm, không được trồng đậu phộng đồng thời ở hai ô kề nhau. Kề nhau ở đây nghĩa là có chung cạnh; chỉ có chung điểm thì không tính là kề. Xiao Keke muốn giúp Sealock chọn các ô trồng sao cho tổng diện tích thực tế trồng được là lớn nhất.

Tương ứng giữa phương án chọn và lát cắt của mạng. Phương pháp lập tương ứng một-một giữa phương án và lát cắt của mạng đã được mô tả trong tài liệu tham khảo số 4. Ta ôn lại theo bài này.

Biến mỗi ô thí nghiệm thành một đỉnh và nối các ô kề nhau, ta nhận được một đồ thị hai phía. Bằng tô màu đen trắng, ta gọi một phần đỉnh là đỉnh trắng và phần còn lại là đỉnh đen. Từ đồ thị hai phía này xây dựng mạng: thêm nguồn và đích; nối nguồn tới mỗi đỉnh trắng bằng cạnh có sức chứa bằng diện tích ô tương ứng; nối mỗi đỉnh đen tới đích bằng cạnh có sức chứa bằng diện tích ô tương ứng. Cuối cùng,

biến các cạnh giữa hai ô kề nhau thành cạnh trong mạng, hướng từ đỉnh trắng sang đỉnh đen, sức chứa là vô cùng.

Lát cắt tương ứng với một phương án được dựng như sau: đưa các đỉnh trắng đã chọn, các đỉnh đen không chọn, cùng với nguồn vào một tập; các đỉnh còn lại vào tập kia. Đảo ngược trực tiếp quá trình này, ta dễ dàng nhận được một phương án từ một lát cắt. Trong tương ứng này, mọi phương án không hợp lệ ứng với lát cắt có sức chứa vô cùng; còn trong lát cắt ứng với phương án hợp lệ, các cạnh bị cắt biểu thị phần diện tích mà phương án không lấy được. Vì vậy, khi sức chứa lát cắt nhỏ nhất, diện tích được chọn là lớn nhất. Theo định lý luồng cực đại - lát cắt cực tiểu, ta có thể tìm lát cắt cực tiểu bằng cách tính luồng cực đại.

Tìm đường tăng bằng BFS. Đây cũng là mạng chuyển hóa từ đồ thị hai phía, nhưng sức chứa cạnh đã được tổng quát hóa. Ta vẫn có thể không tìm hai cạnh đầu và cuối. Điểm khác là tìm kiếm theo chiều rộng bắt đầu từ một “nguồn mới” (đỉnh thứ hai trên đường tăng gốc) có thể phải thực hiện nhiều lần, tối đa bằng sức chứa của cạnh từ nguồn tới nó. Tương tự, một “đích mới” có thể chứa nhiều đường tăng. Ngoài ra, thiết kế riêng quá trình mở rộng cho đỉnh trắng và đỉnh đen cũng có thể nâng hiệu suất thuật toán đáng kể.

3 Cải tiến thuật toán sai: một dạng tối ưu hóa linh hoạt hơn

3.1 Giới thiệu

Ta thường nói tối ưu hóa thuật toán có bốn hướng: tối ưu độ phức tạp thời gian, tối ưu độ phức tạp không gian, tối ưu độ khó cài đặt, và tối ưu độ khó tư duy. Nhưng như tiêu đề phần này đã nói, nội dung ở đây là cách nâng cao tỷ lệ đúng của thuật toán.

Sửa lỗi cũng là tối ưu hóa sao? Nếu bạn có cùng thắc mắc, có lẽ bạn đang nghĩ tới việc dò lỗi trong mã nguồn. Nâng cao tỷ lệ đúng của thuật toán đương nhiên là tối ưu hóa thuật toán. Thậm chí, lỗi thuật toán thường cũng do chưa tận dụng đầy đủ thông tin của đề bài; ngoài ra còn nhiều nguyên nhân khác, lát nữa ta sẽ phân tích thêm.

Đối phó với lỗi cần có phương pháp. Trước hết, ta luôn mong lỗi không xuất hiện, chẳng hạn tư duy chặt chẽ hơn, nhìn vấn đề toàn diện hơn. Khi lỗi không thể tránh khỏi, cách xử lý phải tùy tình huống: nếu thời gian thi không còn nhiều, có thể dùng vài xử lý đơn giản để nâng tỷ lệ đúng; nếu chưa căng thẳng, nên phân tích kỹ lỗi; nếu thật sự không còn khả năng, bỏ thuật toán đang có để tìm lời giải khác cũng là lựa chọn khôn ngoan.

Các xử lý đơn giản trong tình huống khẩn cấp tuy bất đắc dĩ nhưng vẫn đáng nhắc tới. Cách trực tiếp nhất là thêm các nhánh kiểm tra phụ, cố gắng bao phủ các phản ví dụ nghĩ ra; nhưng khi bài toán phức tạp, việc này thường nhúc nhủ và thu được không nhiều. Mặt khác, ngẫu nhiên hóa cộng với giải lặp lại là một xử lý đơn giản và hiệu quả đáng ngạc nhiên, nên dễ được chấp nhận hơn. Với bài toán tối ưu, chỉ cần lấy lời giải tốt nhất trong nhiều lần chạy. Với bài toán quyết định thì phiền hơn một chút; thuật toán ban đầu còn phải thỏa ít nhất một trong hai tiền đề: hoặc tỷ lệ đúng của nó cao hơn 50%, khi đó ta lấy kết quả xuất hiện nhiều hơn; hoặc nó phán đoán chính xác một phía của “có” và “không”. Nếu khi trả lời “có” chắc chắn đúng, thì những đầu vào mà mọi lần còn lại đều trả lời “không” sẽ được xem là “không”. Khi xử lý lỗi cụ thể còn có thể tìm ra nhiều mẹo nhỏ; ở đây không liệt kê thêm.

Dưới đây, tôi sẽ dùng phân loại kết hợp ví dụ để trình bày cách tôi hiểu việc phân tích lỗi cần thận. Nguyên nhân khiến ta thiết kế ra một thuật toán sai đại khái có ba loại:

1. Hiểu sai tính chất của mô hình. Mô hình ở đây bao gồm mọi mô hình toán học; việc chưa tận dụng đầy đủ thông tin cũng thuộc loại này. Đây là nguyên nhân chủ đạo, sẽ được bàn sâu hơn trong bài

Fishing Net.

2. Phỏng đoán sai. Khi chưa biết liên hệ mô hình với thuật toán thế nào, ta thường khuyến khích mạnh dạn phỏng đoán. Nhưng phỏng đoán khó tránh khỏi sai; xử lý loại lỗi này rất quan trọng. Vấn đề này xuất hiện trong phân tích bài *Flying Right*.
3. Chưa nắm chi tiết thuật toán. Khi cần cải tiến một thuật toán kinh điển, nếu ta nắm thuật toán ấy chưa đủ thấu đáo thì rất dễ phát sinh vấn đề. Ví dụ *Cow Patterns* phía dưới không phải bài đồ thị, nhưng rất tiêu biểu; nó chủ yếu liên quan tới thuật toán KMP.

3.2 Ví dụ: Flying Right

Những con bò của John mở một tuyến hàng không phục vụ riêng cho bò. Mỗi sáng, chúng bay dọc bờ tây hồ Michigan từ điểm cực bắc xuống điểm cực nam, qua N sân bay, kể cả hai đầu. Buổi chiều, chúng bay theo cùng tuyến đường về lại điểm cực bắc. Mỗi ngày có K nhóm bò khác nhau muốn đi máy bay; một nhóm bò chờ ở một sân bay và muốn bay tới một sân bay cụ thể khác.

Máy bay chỉ chở đồng thời tối đa C hành khách bò. Con bò phụ trách chuyến bay muốn biết trong ngày đó chúng có thể thỏa mãn yêu cầu của nhiều nhất bao nhiêu con bò. Máy bay có thể chỉ chở một phần của một nhóm bò tới đích. Ràng buộc là

$$1 \leq N \leq 10000, \quad 1 \leq K \leq 50000, \quad 1 \leq C \leq 100.$$

Phân tích ban đầu. Lướt đi và lướt về rõ ràng là hai bài toán giống nhau. Nếu một con bò nhất định phải vòng qua điểm quay đầu, nó sẽ ngồi trên máy bay thêm một đoạn so với phương án trực tiếp, không hề có lợi. Vì vậy hai bài con độc lập, có thể xử lý riêng; ta chỉ cần xét một hướng. Cũng không cần bận tâm máy bay dừng ở những sân bay nào trên đường. Có thể xem như nó dừng ở mọi sân bay, hoặc đơn giản coi nó là một xe khách đường dài.

Cảm giác đầu tiên là quy hoạch động: xem sân bay là đỉnh, tuyến bay của bò là cạnh, thuật toán ghi lại tại mỗi đỉnh số bò tối đa có thể phục vụ từ điểm đầu tới đó, rồi chuyển trạng thái theo cạnh. Nhưng liên hệ với tình huống thực tế, ta nhanh chóng thấy ý tưởng này rất vô lý. Trong phương án thu được từ quy hoạch động kiểu đó, chắc chắn không có các cạnh chồng lấn nhau; trong khi máy bay hoàn toàn có thể đồng thời thỏa mãn yêu cầu của hai con bò có lộ trình chồng lấn. Có vẻ đây không phải một bài quy hoạch động thông thường, nên ta tạm gác ý tưởng quy hoạch động lại.

Mô hình luồng chi phí nhỏ nhất. Giới hạn dữ liệu nhắc ta tận dụng điều kiện số ghế trên máy bay không nhiều, tối đa 100. Xét từng ghế một, cách trực tiếp nhất không gì khác ngoài luồng mạng.

Xây mạng sức chứa không phức tạp: vẫn lấy sân bay làm đỉnh, tuyến bay của các nhóm bò làm cạnh. Mỗi cạnh có sức chứa bằng số bò M_i trong nhóm, trọng số bằng 1. Điều này biểu thị nhóm ấy nhiều nhất có thể “chứa” M_i ghế; mỗi khi một ghế chọn nhóm này thì nhận được 1 đơn vị giá trị. Ngoài ra, từ mỗi đỉnh, trừ đỉnh cuối, nối tới đỉnh kế tiếp bằng một cạnh có sức chứa vô cùng và trọng số 0, biểu thị ghế có thể để trống trên đoạn này. Nhìn chung, một đơn vị luồng trong mô hình đại diện cho một ghế.

Để tránh thuật ngữ hơi lạ “luồng chi phí lớn nhất”, khi cài đặt chỉ cần đổi trọng số thành -1 . Trong thuật toán luồng chi phí nhỏ nhất, dùng Bellman–Ford tìm một đường ngắn nhất tốn $\mathcal{O}(KN)$, lượng luồng cực đại bằng số ghế C , nên tổng độ phức tạp là $\mathcal{O}(KNC)$, không chịu nổi quy mô dữ liệu của đề.

Thêm tư tưởng tham lam. Áp nguyên xi thuật toán luồng chi phí nhỏ nhất vào đồ thị đặc biệt này quả là lãng phí, và độ phức tạp cao là tất yếu. Vậy tận dụng các điểm đặc biệt để tối ưu thế nào?

Trong thuật toán luồng chi phí, mỗi lần ta tìm một đường ngắn nhất, tức đường có tổng trọng số nhỏ nhất, để tăng luồng. Khi tăng, để có khả năng điều chỉnh về sau, ta tăng sức chứa của cạnh ngược tương ứng; xử lý này giải quyết vấn đề hậu hiệu. Nghĩ kỹ trong bài này, có lẽ hoàn toàn không tồn tại hậu hiệu: liệu ta có thể bỏ xử lý cạnh ngược không?

Không xử lý cạnh ngược làm thuật toán thay đổi về bản chất. Nó tương đương với việc lần lượt sắp xếp từng ghế, sao cho trong tình hình hiện tại, ghế ấy chở được số bò lớn nhất trên toàn hành trình. Với việc này, không cần Bellman–Ford nữa; quy hoạch động có thể cho đáp án trong $\mathcal{O}(K)$. Tổng thời gian giảm xuống $\mathcal{O}(KC)$, đủ đáp ứng giới hạn đề.

Nhưng thuật toán này có phản ví dụ. Giả sử có bốn tuyến A, B, C, D , mỗi tuyến chỉ có một con bò, được bố trí như trong hình phản ví dụ của bản gốc: máy bay có hai ghế trống thì có thể phục vụ cả bốn con bò. Tuy nhiên thuật toán tham lam trên có thể sắp A và D cho ghế thứ nhất, vì không tồn tại phương án nào chở ba con bò trên một ghế. Khi đó ghế thứ hai dù thế nào cũng chỉ có thể đưa một con bò tới đích. Thuật toán này thật sự chỉ bảo đảm kết quả khá tốt, không nhất thiết tối ưu.

Dù vậy, đây vẫn là thuật toán có tỷ lệ đúng cao. Trong thử nghiệm thực tế, nó qua được 15 trong 16 bộ dữ liệu chính thức.

Đánh giá sâu hơn quan hệ giữa các cạnh. Tiếp tục phân tích thuật toán trước. Với phương án của một ghế, nếu có hai cạnh mà chọn cạnh nào cuối cùng cũng cho cùng tổng số bò ngồi trên ghế ấy, thuật toán sẽ coi hai cạnh tốt như nhau. Nhưng khi ấy giữa chúng vẫn có thể có quan hệ hơn kém. Vì vậy ta cần đánh giá sâu hơn quan hệ tốt xấu giữa các cạnh.

Quan hệ này không dễ phán đoán, vì mỗi cạnh có hai tham số: điểm xuất phát và điểm kết thúc. Một cách xử lý phổ biến cho loại bài này là làm cho một tham số có thứ tự, nhưng ở đây có vẻ không phù hợp. Đổi góc nhìn: nếu tại một sân bay có hai con bò tranh một ghế, ta nên nhường ghế cho con nào? Dĩ nhiên là con có đích gần hơn. Cách nghĩ ấy tự nhiên loại bỏ yếu tố điểm xuất phát: chúng lên từ đâu không ảnh hưởng tới quyết định này.

Từ đó ta có một mô hình tham lam mới: xử lý từng trạm. Khi bay tới một sân bay, xét các con bò muốn lên máy bay cùng các con bò đã ở trên máy bay, chọn C con có đích gần nhất để lên hoặc tiếp tục ngồi, rồi cùng bay tới trạm kế tiếp. Một số con bò có thể bị đuổi xuống giữa đường; dù vậy thuật toán này không nghi ngờ gì là đúng và ngắn gọn.

Dùng mảng ghi các con bò trên máy bay, tại mỗi trạm cập nhật thành viên một lần trong $\mathcal{O}(C)$, ta nhận được thuật toán hiệu quả $\mathcal{O}(NC)$. Một số kỹ thuật cài đặt còn có thể tối ưu nó xuống $\mathcal{O}(N)$.

Nhìn lại và gợi ý. Nhìn lại toàn bộ quá trình: thử quy hoạch động thất bại, ta lùi về một điểm xuất phát thấp hơn là thuật toán luồng mạng. Trong quá trình tối ưu, đầu tiên ta thêm tư tưởng tham lam vào thuật toán luồng mạng. Sau khi phát hiện phản ví dụ, ta phân tích vấn đề nằm ở đâu, điều chỉnh góc độ tham lam, rồi nhận được một thuật toán đúng và hiệu quả hơn. Cuối cùng, ta sẽ thấy đây vẫn là một thuật toán tương tự quy hoạch động, chỉ là dùng tham lam khi chọn trạng thái tối ưu. Dù ban đầu đã nghĩ tới quy hoạch động, ta vẫn lúng túng trước nhiều chi tiết. Vì vậy ta lấy thuật toán luồng mạng chuẩn làm nền tảng, mạnh dạn phỏng đoán, rồi sau khi thấy phỏng đoán sai thì phân biệt rõ vấn đề và thử cải tiến, cuối cùng thu được phương pháp quy hoạch động cụ thể.

Khi đường nghĩ bị chặn, ta cần những phỏng đoán như vậy. Nếu phỏng đoán đúng, thiết kế thuật toán được tiếp tục; ngay cả khi sai, thường cũng không cần bỏ toàn bộ kết quả đã có. Một phỏng đoán sai ít nhất cũng làm một phần bài toán rõ hơn; phân tích và sửa nó, ta vẫn có thể nhận được kết quả thỏa đáng. Năng lực phân tích lỗi là bảo đảm cho việc mạnh dạn phỏng đoán.

3.3 Ví dụ: Fishing Net

Lỗ trên lưới càng nhỏ thì bắt được càng nhiều cá. Vì vậy, sau khi đánh cá trở về, ngư dân phải kiểm tra lưới xem có lỗ lớn không để vá trước lần ra khơi tiếp theo.

Lưới được xem đơn giản như một đồ thị gồm đỉnh và cạnh. Nếu trong đồ thị, mọi chu trình có độ dài lớn hơn 3 đều có ít nhất một đoạn lưới ở giữa nối hai đỉnh trên chu trình nhưng không thuộc chu trình, thì lưới là hoàn hảo. Hãy phán đoán lưới nhập vào có hoàn hảo hay không.

Phân tích ban đầu. Nhắc tới lưới đánh cá, ta rất dễ nghĩ đồ thị cho trước là đồ thị phẳng. Nhưng đọc kỹ mô tả của đề sẽ thấy đồ thị nhận được từ lưới thực ra là tùy ý.

Đây là bài toán chuẩn kiểm tra đồ thị dây cung, tức kiểm tra một đồ thị có phải chordal graph hay không. Đề bài đã giải thích trực quan khái niệm này; ta nêu lại chính xác hơn: nếu trong một đồ thị, mọi chu trình độ dài lớn hơn ba đều có ít nhất một dây cung, thì đồ thị đó là đồ thị dây cung. Dây cung là một cạnh nối hai đỉnh không kề nhau trên chu trình. Để tiện trình bày, chu trình có dây cung được gọi tắt là chu trình hợp lệ, và đường đi có dây cung được gọi tắt là đường đi hợp lệ.

Đã có một thuật toán kiểm tra đồ thị dây cung rất đẹp về hình thức, độ phức tạp $\mathcal{O}(M)$, đủ để giải bài này. Nhưng trong thi tin học, bài về đồ thị dây cung cực kỳ hiếm; thuật toán ấy dựa trên một biến đổi toán học tinh tế và rất chuyên biệt, nên đối với ta giá trị thực dụng không lớn. Ý tưởng và cài đặt cụ thể được mô tả trong tài liệu tham khảo số 1; bài viết này không tập trung vào nó. Dưới đây ta bàn một thuật toán trực quan hơn.

Dùng tìm kiếm để liệt kê chu trình. Theo cách hiểu trực tiếp nhất về định nghĩa đồ thị dây cung, ta nên tìm các chu trình trong đồ thị rồi kiểm tra chúng. Một đồ thị tùy ý không có tính chất tốt nào để tận dụng, nên tìm chu trình trực tiếp trong đó sẽ mù quáng và kém hiệu quả, chẳng hạn liệt kê độ dài rồi liệt kê dây đỉnh có độ dài ấy. Ta nên đơn giản hóa cấu trúc dữ liệu trước.

Thuật toán tìm kiếm cơ bản có thể làm việc này: nó tạo thứ tự cho đồ thị và thu được cây tìm kiếm. Cây không có chu trình, còn chu trình lại chính là thứ ta tìm. Điều này nhắc ta chuyển chú ý sang các cạnh ngoài cạnh cây.

Trước khi nghĩ tiếp, hãy giải quyết một vấn đề khác: DFS hay BFS? Lựa chọn này ảnh hưởng trực tiếp tới cách phán đoán về sau. Thử bằng tay sẽ thấy các cạnh ngoài cây xuất hiện dưới dạng cạnh ngược trong cây DFS, và dưới dạng cạnh ngang trong cây BFS. Khác biệt này khó cho ta kết luận thật chắc chắn, nhưng ít nhất bề ngoài thì quan hệ tổ tiên - hậu duệ do cạnh ngược nối có vẻ đơn giản hơn quan hệ anh em do cạnh ngang nối. Vì vậy, để tiếp tục thảo luận, ta chọn tìm kiếm theo chiều sâu.

Đến đây ta có phác thảo thuật toán: duyệt DFS trên đồ thị; khi phát hiện cạnh ngược thì kiểm tra chu trình liên quan tới cạnh ấy.

Kiểm tra chu trình bằng truy hồi. Nếu gọi một cạnh ngược tương ứng với chu trình hợp lệ là cạnh ngược hợp lệ, thì vấn đề còn lại là phán đoán một cạnh ngược có hợp lệ hay không. Tự nhiên, ta cần biết khoảng cách giữa hai đầu của cạnh ngược trước khi thêm cạnh ấy là bao nhiêu. Nếu khoảng cách lớn hơn hai thì tìm được một chu trình không hợp lệ. Thực ra ta không cần tính khoảng cách cụ thể; chỉ cần biết trong các tổ tiên của một đỉnh, đỉnh nào cách nó một cạnh và đỉnh nào cách nó hai cạnh. Việc này cần hai mảng boolean hai chiều để ghi lại, trong đó mảng đầu tiên về bản chất là một phần của ma trận kề.

Khi cài đặt thuật toán, ta có thể dùng truy hồi để duy trì nhanh mảng thứ hai.

Sắp xếp thuật toán chính. Toàn bộ thuật toán như sau. Duyệt DFS trên đồ thị. Mỗi khi tới một đỉnh chờ mở rộng, trước hết dùng cha của nó để khởi tạo khoảng cách từ nó tới mọi tổ tiên; sau đó lần lượt

kiểm tra mọi cạnh ngược đi ra từ nó, đồng thời cập nhật khoảng cách giữa nó và các tổ tiên. Khi các khoảng cách đã xác định, kiểm tra lại mọi cạnh ngược. Nếu có cạnh ngược nối hai đỉnh có khoảng cách lớn hơn hai, kết luận đồ thị không phải đồ thị dây cung và dừng tìm kiếm. Nếu không, bắt đầu mở rộng các hậu duệ của đỉnh hiện tại.

Do mỗi lần kiểm tra cạnh ngược cần cập nhật khoảng cách từ đỉnh hiện tại tới các tổ tiên, chi phí là $\mathcal{O}(N)$. Số cạnh ngược ở cấp $\mathcal{O}(M)$, nên tổng độ phức tạp là $\mathcal{O}(MN)$. Với dữ liệu $N \leq 1000, M = \mathcal{O}(N^2)$, độ phức tạp này không chấp nhận được. Nhưng vì vòng lặp trong thao tác trên mảng boolean, ta có thể nén không gian thích hợp: lưu 32 biến boolean trong một số nguyên 32 bit. Điều này không thay đổi độ phức tạp thời gian, nhưng trên thực tế có thể giảm thời gian chạy xuống gần một phần mười.

Nhiều hình thái của chu trình trong cây tìm kiếm. Ở phần trước, ta nhận được một thuật toán mới có hiệu suất gần $\mathcal{O}(M)$ trên quy mô dữ liệu của đề. Nhưng suy xét kỹ sẽ thấy trong quá trình xây dựng thuật toán có vài chỗ chưa chặt chẽ.

Trước hết nói một vấn đề nhỏ. Với một chu trình dài, nếu còn có một số đỉnh và cạnh gián tiếp nối các đỉnh không kề nhau trên chu trình, sẽ xuất hiện chu trình ngắn hơn; nếu không, bản thân nó sẽ được thuật toán tìm ra và phán đoán là không hợp lệ. Vì vậy ta tạm bỏ qua chu trình dài, lấy chu trình không hợp lệ ngắn nhất C_4 , tức chu trình không dây cung độ dài 4, làm ví dụ để bàn cách cải tiến.

C_4 có ba hình thái khi xuất hiện trong cây DFS. Thuật toán hiện tại chỉ phán đoán được hình thái thứ nhất, vì trong hai hình thái còn lại, có hơn một cạnh của chu trình không nằm trên cây tìm kiếm. Có hai cách giải quyết: thêm quy trình phán đoán riêng, hoặc cố gắng thống nhất ba trường hợp rồi xử lý.

Sau vài thử nghiệm đơn giản, xử lý riêng hai trường hợp còn lại có khó khăn nhất định và có thể khiến thuật toán khó cài đặt. May mắn là có một cách đơn giản để thống nhất các trường hợp: ngẫu nhiên hóa. Có thể xem ba hình thái xuất hiện với xác suất bằng nhau; dùng tổ hợp đơn giản sẽ nhận được bảng sau:

Số lần ngẫu nhiên hóa	1	2	3	4	5	6
Xác suất bỏ sót một C_4 cụ thể	66.7%	44.4%	29.6%	19.8%	13.2%	8.8%

Dễ thấy sau vài lần giải lặp, ngay cả khi đồ thị chỉ có một chu trình không hợp lệ, xác suất bỏ sót nó đã rất nhỏ. Trong cài đặt cụ thể, chọn trên 5 lần là đủ đáp ứng yêu cầu của đề.

Sự che khuất giữa các chu trình. Thuật toán trên đã vượt qua 9 bộ dữ liệu trên ZOJ và gần như không sai với đồ thị ngẫu nhiên. Nhưng đáng tiếc là phản ví dụ vẫn tồn tại. Trong thảo luận trước, ta đã bỏ qua việc một cạnh ngược có thể tương ứng với nhiều hơn một chu trình.

Khi tính khoảng cách, ta nhận được độ dài của chu trình ngắn nhất tương ứng sau khi bỏ cạnh ngược. Ngay cả nếu chu trình ấy hợp lệ, vẫn có thể tồn tại các chu trình dài hơn khác. Vì vậy hiệu quả thực tế là: khi thuật toán phán đoán một cạnh ngược không hợp lệ, trong đồ thị chắc chắn có chu trình không hợp lệ; nhưng khi thuật toán phán đoán một cạnh ngược hợp lệ, nó vẫn có thể không hợp lệ. Điều ta kiểm tra chỉ là điều kiện cần để cạnh ngược hợp lệ.

Cải tiến bằng ngẫu nhiên hóa và giải nhiều lần cũng có tác dụng nhất định với lỗi hổng này. Nhưng khi mật độ cạnh của đồ thị gần 1, thuật toán có thể phải ngẫu nhiên rất nhiều lần mới tìm được vài chu trình không hợp lệ hiếm hoi.

Lỗi hổng tồn tại vì điều ta cần không phải độ dài đường đi ngắn nhất giữa hai đầu cạnh ngược; điều ta cần là “độ dài của đường đi không dây cung dài nhất giữa hai đầu ấy”. Nếu độ dài này không vượt quá 2, mọi chu trình tương ứng với cạnh ngược đó đều hợp lệ; ngược lại, ít nhất có một chu trình không hợp lệ.

Không may là suy nghĩ thêm cho thấy dùng truy hồi trên cây không thể tính độ dài đường đi không dây cung dài nhất trong cây DFS. Cách chuyển hướng và tìm độ dài ấy vẫn còn đang được khám phá.

Một cải tiến chưa giải quyết triệt để. Ban đầu, để tránh xuất hiện đường đi có dây cung, ta trực tiếp dùng cách tìm đường đi ngắn nhất. Nhưng đó thực ra lại là đường đi không dây cung mà ta ít cần nhất; thứ tốt hơn là độ dài đường đi không dây cung dài nhất. Thay vì vất vả tìm một đường đi ít hữu dụng như vậy, ta có thể tùy ý tìm một đường đi, miễn là nó không có dây cung.

Thực ra, chỉ cần khi tìm kiếm tới một đỉnh, quét một lần từ dưới lên, ta có thể thu được độ dài của một đường đi không dây cung từ nó tới mỗi tổ tiên. Phương pháp cụ thể không phức tạp; bản gốc đặt chương trình trong phụ lục nên không bàn sâu ở đây. Cải tiến này vừa nâng tỷ lệ đúng vừa giảm độ phức tạp xuống $\mathcal{O}(N^2 + M)$.

Kết quả. Đáng tiếc là kết quả cuối cùng vẫn chưa hoàn hảo. Có thể tin rằng nếu tiếp tục suy nghĩ và thử nghiệm, thuật toán này còn có thể được làm nhanh hơn và tốt hơn.

3.4 Ví dụ: Cow Patterns

Trong đàn bò của John có K , $1 \leq K \leq 25000$, con đặc biệt thích gây chuyện; khi xếp hàng chúng luôn đứng cạnh nhau. Để tìm ra chúng, John cho tất cả N , $1 \leq N \leq 100000$, con bò xếp thành một hàng đi vào chuồng, và muốn biết các nhóm đáng nghi có độ dài K trong hàng.

John phân biệt bò bằng số đếm S , $1 \leq S \leq 25$. Anh ta đã quên chính xác số đếm trên những con bò gây chuyện, nhưng vẫn nhớ con nào nhiều đếm hơn, hoặc những con nào có cùng số đếm. Vì vậy anh ta dùng một dãy thứ hạng số đếm để mô tả nhóm bò ấy. Ví dụ, dãy 1, 4, 4, 3, 2, 1 biểu thị: con thứ nhất và con cuối có số đếm bằng nhau và ít hơn mọi con khác; con thứ năm có nhiều đếm hơn một chút; còn con thứ hai và thứ ba có nhiều đếm nhất. Hãy giúp John tìm tất cả các dãy con phù hợp với mô tả này.

Phân tích ban đầu. Đây là bài khớp mẫu trong đó mẫu có thể dao động. Quy mô không nhỏ, nên để đoán thuật toán cuối cùng sẽ liên quan tới KMP.

Áp trực tiếp chắc chắn không được: hiệu quả của KMP nằm ở chỗ sau khi mẫu dịch sang phải, nó vẫn giữ được kết quả khớp trước đó. Trong bài này, việc dịch vị trí của mẫu rất có thể làm bản thân mẫu thay đổi, nên kết quả cũ rõ ràng không dùng lại được.

Từ liệt kê toàn cục sang liệt kê từng phần. Phân tích trên cho ta một điều đơn giản: mẫu trong KMP phải luôn bất biến; muốn dùng KMP thì trước hết phải xác định mẫu. Điều này gợi ý liệt kê trước rồi khớp từng mẫu một. Nhưng ngay cả khi số đếm tối đa chỉ là 25, cách ấy vẫn không thể chấp nhận. Trường hợp xấu nhất, chọn 12 hoặc 13 số đếm trong 1..25 để ứng với các thứ hạng đã cho hơn năm triệu mẫu cụ thể; khớp từng mẫu trong giới hạn 1 giây là quá chậm.

Việc liệt kê này có thể cắt tĩa, vì đa số trường hợp chỉ cần xác định vài thứ hạng đã thấy mẫu không thể khớp với chuỗi gốc. Thuật toán mới thay đổi khá nhiều: mỗi lần liệt kê số đếm tương ứng với một thứ hạng, dùng KMP để tìm các vị trí trong chuỗi gốc mà mẫu chưa hoàn chỉnh này có thể khớp. Lưu ý rằng dù mẫu chưa hoàn chỉnh, nó vẫn xác định: nếu thứ hạng đang xét ứng với số đếm 5, thì các vị trí của thứ hạng khác trong mẫu luôn khớp với mọi số đếm khác 5, nên vẫn dùng được KMP. Sau đó kiểm tra từng vị trí trong chuỗi gốc: nếu vị trí ấy khớp thành công với mọi thứ hạng, và các số đếm ứng với các thứ hạng đúng theo quan hệ thứ tự, thì dãy con độ dài K bắt đầu tại vị trí đó là dãy đáng nghi cần tìm.

Ví dụ, chuỗi gốc là 45858, thứ hạng của dãy đáng nghi là 121. Trước hết xét thứ hạng 1: khi nó ứng với 5, mẫu 5?5 khớp ở vị trí thứ hai; khi ứng với 8, mẫu 8?8 khớp ở vị trí thứ ba; các số đếm khác không khớp ở đâu. Sau đó xét thứ hạng 2: khi ứng với 5, mẫu ?5? khớp ở vị trí 1 và 3; khi ứng với 8, mẫu ?8?

chỉ khớp ở vị trí thứ hai. Tổng hợp:

Chuỗi gốc	4	5	8	5	8
Khớp thứ hạng 1	–	5	8	–	–
Khớp thứ hạng 2	5	8	5	–	–

Vị trí 2 và 3 đều khớp thành công với hai thứ hạng, nhưng vị trí 3 không thỏa quan hệ lớn nhỏ của thứ hạng, nên cuối cùng chỉ có một dãy con đáng nghi.

Bỏ qua các vị trí đã bị loại. Thuật toán trên đã có bước nhảy vọt về thời gian chạy, nhưng vẫn chưa hoàn toàn đạt yêu cầu của đề.

Nó vẫn làm nhiều việc vô ích. Trong ví dụ trên, thứ hạng thứ nhất không thể khớp các vị trí 1, 4, 5, vậy khi xét thứ hạng thứ hai nên trực tiếp bỏ qua chúng. Nhìn xa hơn, nếu xử lý các thứ hạng theo thứ tự lớn nhỏ, mâu thuẫn về quan hệ lớn nhỏ cũng cho phép ta yên tâm bỏ qua một số vị trí.

Nhưng trong thao tác cụ thể, làm thế nào để “bỏ qua” một vị trí? Khớp xong một vị trí rồi tìm vị trí còn ý nghĩa tiếp theo để bắt đầu khớp? Như vậy chính là khớp mẫu ngẫu thơ. Ta vẫn phải dùng tư tưởng của KMP: nếu vị trí hiện tại đã không còn ý nghĩa, tiếp tục dùng hàm hậu tố để đẩy mẫu sang phải.

Một tối ưu nhỏ của KMP. Tối ưu tới bước này, ta đã có một thuật toán hiệu quả. Điều lạ là nó vẫn sai trong kiểm thử. Trước khi phân tích lỗi, hãy nhớ lại một tối ưu nhỏ trong hàm hậu tố của KMP.

Dùng một ví dụ để giải thích. Giả sử mẫu là aaab, và `next()` biểu thị hàm hậu tố. Khi ký tự a thứ ba khớp thất bại, KMP không tối ưu sẽ đẩy mẫu sang phải một vị trí và thử khớp ký tự a thứ hai, tức `next(3) = 2`. Nhưng lần khớp này rõ ràng cũng thất bại, nên trong trường hợp ấy có thể gán giá trị hàm hậu tố tiến thêm một bước: `next(3) = next(2)`, để tránh so sánh vô ích.

Điều cần chú ý là tối ưu này có một tiền đề ngầm: mỗi lần mẫu bị đẩy sang phải đều do khớp thất bại. Có tiền đề ấy, ta mới xác định được rằng nếu ký tự cần khớp của mẫu sau khi dịch không thay đổi thì lần khớp vẫn sẽ thất bại. Nhưng trong thuật toán trên, ta còn dùng cách đẩy này để bỏ qua các vị trí đã bị loại; lúc đó quá trình khớp có thể vẫn tiếp tục được.

Sau khi bỏ tối ưu này, bài toán cuối cùng được giải quyết trọn vẹn.

Gợi ý. Nếu đưa các chi tiết thuật toán như vậy thành câu hỏi lý thuyết, chỉ cần suy nghĩ một chút có lẽ ai cũng trả lời đúng. Nhưng trong kỳ thi căng thẳng, một chi tiết trước đó chưa từng được cân nhắc lại rất dễ bị bỏ qua. Chỉ sau vài lần sai và đủ nhiều suy nghĩ, ta mới thật sự nắm vững một thuật toán.

3.5 Tiểu kết

So với tối ưu độ phức tạp thời gian, tìm và sửa lỗi thuật toán là một dạng tối ưu hóa linh hoạt hơn: sự nhạy cảm với lỗi đem lại cho ta nhiều cảm hứng hơn. Tích lũy và tổng kết rộng rãi kinh nghiệm sửa lỗi sẽ làm quá trình thiết kế thuật toán trôi chảy hơn, để ta không luôn bế tắc vì một vấn đề nhỏ, thậm chí phải làm lại từ đầu. Trong thế giới thuật toán, có nhiều con đường dẫn tới đích.

4 Kết luận

Mỗi lần tối ưu hóa thuật toán là một chuyến đi của tư duy. Sự cẩn thận và kinh nghiệm là đôi chân của tư duy; thông tin đặc thù là cột mốc trên đường. Dù có đi tới đích hay không, tư duy vẫn thu được điều gì đó.

Tài liệu tham khảo

1. *Chordal Graph Isomorphism*, không rõ tác giả.
2. Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
3. Bruce Merry, phân tích của USACO về bài *Flying Right*.
4. Jiang Peng, *Bàn về cách xây dựng mạng luồng và thuật toán từ lời giải của một bài toán*.

Lời cảm ơn

Tác giả chân thành cảm ơn cô Wu Yani vì sự hướng dẫn và giúp đỡ, đồng thời cảm ơn Zeng Cheng và Wu Bin vì những ý kiến quý báu dành cho bản thảo.

Phụ lục

Bản gốc kèm đề đầy đủ của các bài *Pasture Planning*, *Flying Right*, *Fishing Net*, *Cow Patterns*, cùng một chương trình Pascal cho thuật toán cuối trong phần *Fishing Net*. Ở bản dịch này, các phát biểu bài toán đã được lồng vào phần phân tích tương ứng. Danh sách chương trình Pascal khá dài và chỉ đóng vai trò phụ lục cài đặt, nên không chép lại; ý tưởng thuật toán của nó đã được tóm tắt trong phần *Fishing Net*.